

Activity No. 7.2	
Hands-on Activity 7.2 Sorting Algorithms	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed: 10/21/24
Section: CPE21S1	Date Submitted: 10/23/24
Name(s): Delavin, Katarina Nicole R.	Instructor: Engr. Maria Rizette Sayo
6. Output	
7. Supplementary Activity	
PROBLEM 1: <pre> #include <iostream> void merge(int array[], int left, int middle, int right) { int n1 = middle - left + 1; int n2 = right - middle; int* L = new int[n1]; int* R = new int[n2]; for (int i = 0; i < n1; i++) L[i] = array[left + i]; for (int j = 0; j < n2; j++) R[j] = array[middle + 1 + j]; int i = 0; int j = 0; int k = left; while (i < n1 && j < n2) { if (L[i] <= R[j]) { array[k] = L[i]; i++; } else { array[k] = R[j]; j++; } k++; } while (i < n1) { array[k] = L[i]; i++; k++; } while (j < n2) { </pre>	

```

        array[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

void mergeSort(int array[], int left, int right) {
    if (left < right) {
        int middle = (left + right) / 2;

        mergeSort(array, left, middle);
        mergeSort(array, middle + 1, right);

        merge(array, left, middle, right);
    }
}

int partition(int array[], int left, int right) {
    int pivot = array[right];
    int i = left - 1;

    for (int j = left; j < right; j++) {
        if (array[j] < pivot) {
            i++;
            std::swap(array[i], array[j]);
        }
    }

    std::swap(array[i + 1], array[right]);
    return i + 1;
}

void quickSort(int array[], int left, int right) {
    if (left < right) {
        int pivotIndex = partition(array, left, right);

        if (pivotIndex - 1 > left) {
            mergeSort(array, left, pivotIndex - 1);
        }

        if (pivotIndex + 1 < right) {
            mergeSort(array, pivotIndex + 1, right);
        }
    }
}

int main() {
    int array[] = {4, 12, 19, 25, 29, 30, 34, 43, 44, 48, 93, 67, 43,
19, 74};

```

```

    int size = sizeof(array) / sizeof(array[0]);

    quickSort(array, 0, size - 1);

    std::cout << "Sorted array: ";
    for (int i = 0; i < size; i++) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}

```

Array:

Sorted array: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

[4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74]

Partitioning the Array:

Suppose we choose the last element 74 as the pivot. The array is partitioned so that all elements less than 74 are placed to the left, and all elements greater than 74 are placed to the right. The result would be:

[19, 12, 25, 29, 30, 43, 44, 4, 48, 53, 87, 67, 93, 34, 74]

Sorting Left Subarray:

- The left subarray{4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74} is now small enough to apply other sorting algorithms like Bubble Sort or Insertion Sort.

Sorting Right Subarray:

- Similarly, the right subarray {87, 93, 67} can be sorted using **Insertion Sort**. The sorted result would be:

Merging the Subarrays:

- After sorting the two subarrays, we can combine them with the pivot element:

Left subarray after merge sort: 4 12 19 25 29 30 34 43
 44 48 53 67
 Right subarray after quick sort: 87 93
 Sorted array: 4 12 19 25 29 30 34 43 44 48 53 67 74 87
 93

PROBLEM 2:

Problem 2: Sorting the Array {4, 12, 19, 25, 29, 30, 34, 43, 44, 48, 93, 67, 43, 19, 74} and Choosing the Best Sorting Algorithm

For the given array {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}, Quick Sort is the fastest sorting algorithm in most practical cases, including this one.

What sorting algorithm will give you the fastest time performance?

- With an average-case time complexity of $O(N * \log N)$, Quick Sort is the most effective method for the provided array. It is also typically faster than other algorithms because of its in-place sorting and superior cache efficiency.

Why can merge sort and quick sort have $O(N * \log N)$ for their time complexity?

- Quick Sort and Merge Sort both use the divide-and-conquer strategy, which divides the array into smaller subarrays, to reach $O(N * \log N)$ time complexity. While sorting or merging takes $O(N)$ time at each level of recursion, resulting in an overall time of $O(N * \log N)$, the logarithmic term is produced by periodically splitting the array.

8. Conclusion

Code + Console Screenshot

```
#ifndef SORTING_ALGORITHMS_H
#define SORTING_ALGORITHMS_H

#include <iostream>

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

// Quick Sort implementation (using Lomuto partition scheme)
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
        }
    }
}
```

```

        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

// Shell Sort Implementation (using Knuth's sequence for gaps)
void shellSort(int arr[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}

#endif // SORTING_ALGORITHMS_H

```

Observations

Large data benefit greatly from Quicksort speed and efficiency, but in the worst situation, if the pivot is improperly selected, it may become slow. Although it is easier to use and more efficient for smaller datasets, Shell Sort typically performs slower than Quicksort for bigger arrays. It does this by reducing shifts through the use of a gap approach.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```

// Sorting Algorithms
#ifndef SHELL_SORT_H
#define SHELL_SORT_H

void shellSort(int array[], int size) {
    // Start with a large interval, then reduce the interval
    for (int interval = size / 2; interval > 0; interval = interval / 2) {
        // Perform a gapped insertion sort for this interval size
        for (int i = interval; i < size; i++) {
            int temp = array[i];
            int j = i;
            // Shift elements of array[0..i] that are greater than temp
            // to their new positions
            while (j >= interval && array[j - interval] > temp) {
                array[j] = array[j - interval];
                j -= interval;
            }
            array[j] = temp; // Place temp in its correct location
        }
    }
}

#endif

#include <iostream>

int main(){
    int arr[] = {13, 25, 30, 18, 17, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    shellSort(arr, n);

    std::cout << "Sorted array: \n";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}

```

Sorted array:
5 13 17 18 25 30

Observations

I think Shell Sort outperforms Insertion Sort in terms of sorting speed and fewer shifts. Although it is more effective than Insertion Sort for bigger arrays, it may not be as quick as more sophisticated algorithms like Quick Sort, and its performance still depends on the gap sequence utilized.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    int leftArr[n1], rightArr[n2];

    for (int i = 0; i < n1; i++) {
        leftArr[i] = arr[left + i];
    }

    for (int j = 0; j < n2; j++) {
        rightArr[j] = arr[middle + 1 + j];
    }

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
}
```

```

while (i < n1) {
    arr[k] = leftArr[i];
    i++;
    k++;
}

while (j < n2) {
    arr[k] = rightArr[j];
    j++;
    k++;
}
}

void mergeSort(int arr[], int left, int right) {
    if (left >= right) {
        return;
    }
    int middle = (left + right) / 2;

    mergeSort(arr, left, middle);
    merge(arr, left, middle, right);
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

```

```

    int middle = (left + right) / 2;

    mergeSort(arr, left, middle);
    merge(arr, left, middle, right);
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    const int n = 10;
    int arr[n] = {13, 25, 17, 18, 30, 69, 5, 1, 8, 0};

    cout << "Original Array: ";
    printArray(arr, n);

    mergeSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    printArray(arr, n);

    return 0;
}

```

```
Original Array: 13 25 17 18 30 69 5 1 8 0
Sorted Array: 13 17 18 25 30 69 5 1 8 0
```

Observations

Although its space complexity may be a drawback, the technique is effective and consistently has a time complexity of $O(n \log n)$. Although it works well with huge arrays and is a stable technique, it might use more memory than in-place algorithms like Quick Sort.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {

    int arr[] = {13, 30, 25, 5, 18, 17, 69, 13, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array: ";
    printArray(arr, n);

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array: ";
    printArray(arr, n);
    return 0;
}
```


	Original Array: 13 30 25 5 18 17 69 13 11 Sorted Array: 5 11 13 13 17 18 25 30 69
Observations	The pivot selection has a significant impact on its performance; in the worst scenario, it may deteriorate to $O(n^2)$, but because of its lower memory needs, it frequently performs better than other sorting algorithms like Merge Sort. Quicksort efficiency can be considerably increased by optimizing pivot choices.

Table 8-4. Quick Sort Algorithm

8. Conclusion

In conclusion, our exploration of sorting algorithms, specifically Quick Sort and Merge Sort, has provided valuable insights into their performance characteristics and practical applications. We learned that both algorithms have an average time complexity of $O(N \log N)$, but Quick Sort is generally faster in practice due to its lower overhead and better cache performance, while Merge Sort is stable and performs well with linked lists and nearly sorted data. Through implementing these algorithms in C++ and measuring their execution times, we gained a practical understanding of their performance differences. The activity reinforced the importance of considering the nature of the data when selecting a sorting algorithm, as well as the need to balance theoretical knowledge with hands-on experience. Upon reflection, I believe I performed well in this activity, successfully implementing both algorithms and effectively measuring their performance. However, I identify areas for improvement, including exploring optimizations for Quick Sort, studying additional sorting algorithms, and delving deeper into theoretical complexity analysis. Overall, this activity has solidified my understanding of sorting algorithms and their practical applications, while also highlighting areas for further exploration and improvement.

9. Assessment Rubric